Análisis y Diseño de Algoritmos

Grado de Ingeniería Informática

Universidad de Valladolid

Primera Práctica: Análisis Empírico de Algoritmos de Ordenación

Grupo: 31

Autoras:
Pérez Alonso, Marta
Rodrigues Álvarez, Emily Fabiana

Índice

1. Metodología **2**
- 1.1. Entorno de desarrollo . . . 2
- 1.2. Configuración para la realización de la práctica . . . 2

2. Resultados e interpretaciones **3**
- 2.1. Tablas de datos numéricos . . . 3
  - 2.1.1. Tiempo de ejecución . . . 3
  - 2.1.2. Asignaciones . . . 3
  - 2.1.3. Comparaciones . . . 4
  - 2.1.4. Resumen de los datos observados . . . 4
- 2.2. Gráficas . . . 5
  - 2.2.1. Tiempo de ejecución . . . 5
  - 2.2.2. Asignaciones . . . 6
  - 2.2.3. Comparaciones . . . 7
  - 2.2.4. Resumen de lás gráficas observadas . . . 8

3. Análisis de resultados **9**
- 3.1. Métodos de análisis . . . 9
- 3.2. Fórmulas obtenidas . . . 9
  - 3.2.1. Para `ordenacion1` . . . 9
  - 3.2.2. Para `ordenacion2` . . . 10
  - 3.2.3. Para `ordenacion3` . . . 10
- 3.3. Comparación . . . 10
  - 3.3.1. Eficiencia según el tipo de relleno . . . 10
  - 3.3.2. Análisis de robutez . . . 11

4. Conclusiones **12**

5. Porcentaje de trabajo individual **13**

6. Uso de inteligencia artificial generativa (IA-Gen) **14**
- 6.1. En la estructura del documento . . . 14
- 6.2. En el código . . . 14
- 6.3. En el cálculo de la complejidad . . . 14

7. Bibliografía y Fuentes **15**

1. Metodología

1.1. Entorno de desarrollo

Hemos desarrollado el análisis en `JAVA`. Configuramos además un `Makefile` para:

- Desactivar la compilación en tiempo de ejecución (`JIT`) usando la bandera `-Xint`.
- Ampliar el tamaño de la pila para evitar desbordamientos por la recursión en el algoritmo de `ordenacion2`, usando la bandera `-Xss16m` para que tuviese un tamaño de 16 MB.
- Automatizar la compilación, ejecución y limpieza de archivos binarios.
- Incluir un comando de ayuda para conocer los comandos con los que cuenta.

1.2. Configuración para la realización de la práctica

Para la toma de datos, se han almacenado en distintos archivos `CSV` de manera que, para cada tipo de análisis con su algoritmo de ordenación y relleno correspondiente, generase un único archivo que cuenta con los promedios de cada tamaño.

Los tamaños empiezan con un vector de 10.000 elementos hasta 100.000 con paso de 10.000. Por cada tamaño se realizó el análisis 20 veces reinicializando el vector, y con esos datos, el propio programa calculaba el promedio.

Antes de cada medición de tiempo se realizó una limpieza con el recolector de basura mediante la instrucción `System.gc()`. Y para mejor precisión en nanosegundos, se utilizó (como dice el enunciado) `Instant` y `Duration`, aunque se obtuvieron datos de tiempos muy grandes y decidimos, a partir de esta instrucción, transformarlos a microsegundos.

Y como especificaba la práctica, para el algoritmo de relleno `aleatorio2` el valor de k siempre fue 20.

2. Resultados e interpretaciones

2.1. Tablas de datos numéricos

2.1.1. Tiempo de ejecución

	Ordenación 1		Ordenación 2		Ordenación 3	
Tamaño	***aleatorio1***	***aleatorio2***	***aleatorio1***	***aleatorio2***	***aleatorio1***	***aleatorio2***
10000	198744,55	1282,40	3181,45	23898,40	16904,05	15941,00
20000	783490,05	2369,25	6555,60	91904,30	33844,50	32261,55
30000	1756550,65	3641,70	10216,90	203182,20	51630,65	49138,95
40000	3141313,35	4760,10	13627,60	359399,55	68358,75	64772,20
50000	4860562,20	5890,45	17311,60	554332,15	86216,95	81045,25
60000	7002836,55	7061,15	21077,05	799198,65	105299,60	98551,45
70000	9517332,90	8224,20	25151,60	1077844,45	122727,10	113780,20
80000	12434630,55	9416,55	28579,15	1414770,05	140022,95	131297,30
90000	15717194,50	10573,25	32467,45	1776506,60	157028,95	149308,05
100000	19418385,75	11763,40	36229,10	2180658,00	174843,55	165880,65

Tabla 1: Tiempo de ejecución promedio (Microsegundos, μs)

Con `ordenacion1` y `aleatorio1` obtenemos tiempos 150-200 veces superiores a su versión con aleatorio2. Para 100.000 elementos, la diferencia es extrema: 19.418µs vs 11,7µs.

Por su parte `ordenacion3` mantiene diferencias mínimas entre rellenos (máximo 9 % de diferencia), demostrando robustez.

Para el caso de `ordenacion2` es 7-60 veces más lento con a`aleatorio2`, a diferencia del resto.

2.1.2. Asignaciones

	Ordenación 1		Ordenación 2		Ordenación 3	
Tamaño	***aleatorio1***	***aleatorio2***	***aleatorio1***	***aleatorio2***	***aleatorio1***	***aleatorio2***
10000	25068940,70	103835,95	102412,65	49784,70	124159,45	71658,85
20000	99958049,75	208452,15	218458,80	99490,65	262430,00	143295,15
30000	224759909,75	312598,70	339838,65	149305,65	405247,30	214945,25
40000	400164679,90	416522,75	465348,00	199021,65	553361,05	286432,15
50000	624752048,50	521023,60	591686,40	248679,60	702342,75	358318,00
60000	900464881,25	624433,80	721267,35	298543,80	852288,20	430321,55
70000	1224938375,75	729591,25	851564,10	348274,20	1004306,00	501631,45
80000	1599524062,10	833514,95	981672,30	398210,25	1162888,45	573580,65
90000	2025216394,40	937660,60	1116842,10	447814,35	1318607,65	645288,65
100000	2501454446,05	1041547,65	1251767,10	497587,20	1469810,00	716353,95

Tabla 2: Asignaciones promedio

Vislumbramos que de manera general, el algoritmo de relleno de vector que produce el menor número de asignaciones es `aleatorio2`.

Con el algoritmo `ordenacion1` y `aleatorio1` se generan volúmenes masivos de asignaciones (2.500 millones para 100 mil elementos), mientras que con `aleatorio2` se reduce drásticamente a solo 1 millón.

En cambio, `ordenacion2` es consistentemente el algoritmo que menos asignaciones requiere, especialmente con `aleatorio2`.

El que muestra un crecimiento moderado entre los tres es `ordenacion3`.

2.1.3. Comparaciones

	Ordenación 1		**Ordenación 2**		**Ordenación 3**	
Tamaño	***aleatorio1***	***aleatorio2***	***aleatorio1***	***aleatorio2***	***aleatorio1***	***aleatorio2***
10000	24940785,60	94163,45	203350,80	5622193,30	197476,65	193176,30
20000	99983956,20	188357,65	435760,10	22215804,50	419556,35	420247,95
30000	224930265,55	282734,05	681934,50	50353082,00	652979,35	644727,20
40000	399772296,70	376410,60	947049,35	89222368,20	883581,80	896461,10
50000	624787557,35	470852,50	1205988,30	138888894,45	1135064,30	1134958,10
60000	899891738,60	564990,10	1470327,50	199798164,00	1377135,85	1383229,05
70000	1225337807,05	659641,50	1739130,45	272520316,85	1646381,10	1654596,20
80000	1600767605,00	753951,95	2002878,95	355792491,45	1908908,50	1886840,70
90000	2024906898,95	847803,10	2258586,25	449853902,10	2154461,70	2143409,15
100000	2501261707,10	941823,75	2575285,60	555640948,90	2413318,45	2423895,85

Tabla 3: Comparaciones promedio

Se tiene el contraste más extremo en `ordenacion2`, debido a que, con `aleatorio1` se realizan solo 2,5 millones de comparaciones para 100 mil elementos, pero con `aleatorio2` se alcanzan 555 millones de comparaciones.

Sin embargo, con `ordenacion1` con `aleatorio2` se reducen las comparaciones en un 99,6 % respecto a `aleatorio1`.

Y como se ha ido viendo, `ordenacion3` mantiene valores muy similares entre ambos rellenos, con diferencias inferiores al 0,5 %.

2.1.4. Resumen de los datos observados

De manera global podemos concluir que con los datos observados en las tres tablas, `ordenacion1` cuenta con valores extremos según relleno, de manera que es ineficiente con `aleatorio1` pero competitivo con `aleatorio2`.

Por su parte, `ordenacion2` tiene un comportamiento más irregular en comparación con los demás algoritmos de ordenación, siendo que sufre más con `aleatorio2` que con `aletorio1` en la mayoría de las medidas.

Quien maneja valores más consistentes y de manera general es el más robusto de todos es `ordenacion3`, contando con diferencias mínimas entre ambos tipos de relleno.

Igualmente, es `ordenacion1` quien de manera general contiene los valores más extremos.

También pudimos ver como `aleatorio2` genera mejores resultados en las mayoría de las métricas y tipos de ordenación, menos para `ordenacion2` y sus análisis de tiempo y comparaciones.

2.2. Gráficas

2.2.1. Tiempo de ejecución

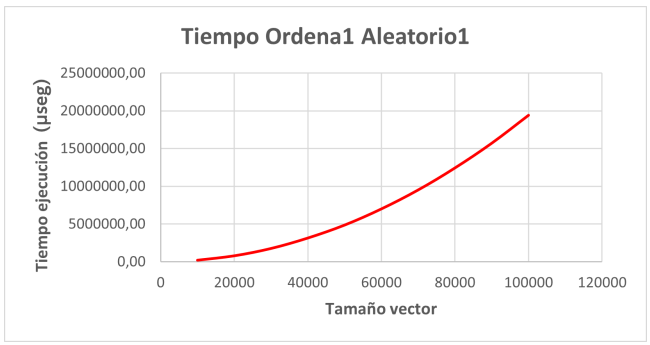




Figura 1: Gráficas de `ordenacion1`





Figura 2: Gráficas de `ordenacion2`





Figura 3: Gráficas de `ordenacion3`

2.2.2. Asignaciones





Figura 4: Gráficas de `ordenacion1`

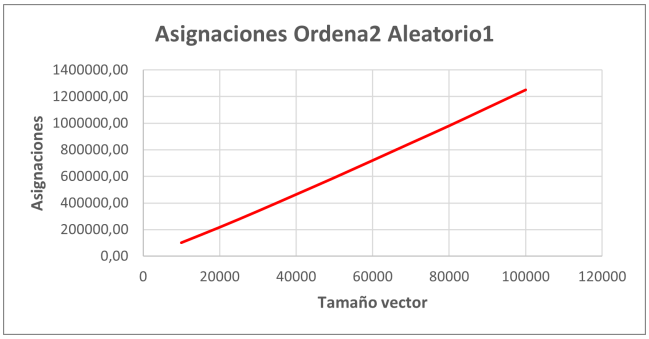




Figura 5: Gráficas de `ordenacion2`



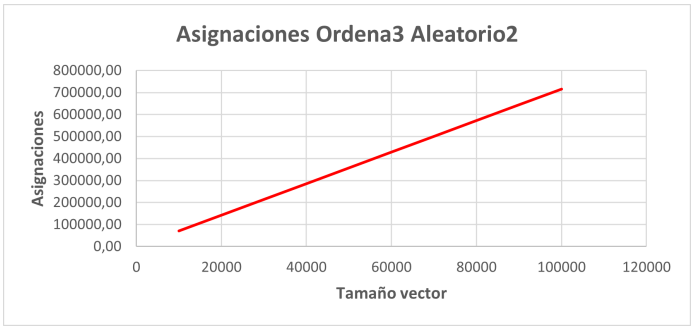


Figura 6: Gráficas de `ordenacion3`

2.2.3. Comparaciones



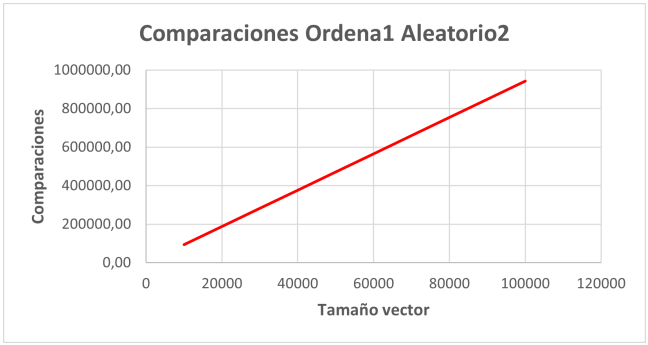
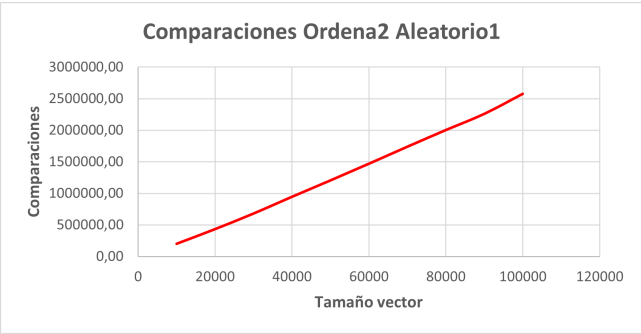


Figura 7: Gráficas de `ordenacion1`



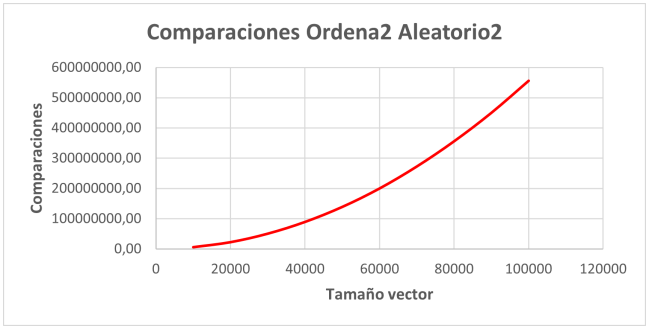


Figura 8: Gráficas de `ordenacion2`

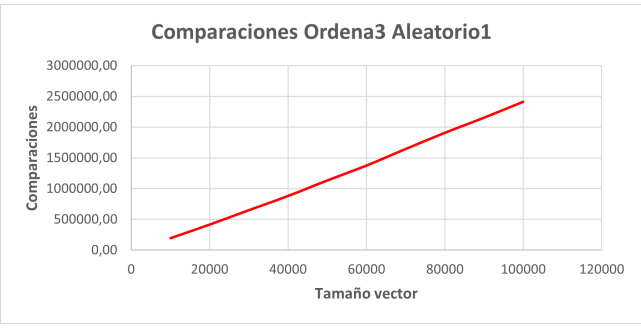




Figura 9: Gráficas de `ordenacion3`

2.2.4. Resumen de lás gráficas observadas

Las gráficas confirman visualmente los patrones identificados en las tablas.

Para `ordenacion1` con `aleatorio1` muestra crecimiento cuadrático evidente en todas las métricas.

Por otro lado, `ordenacion2` presenta el comportamiento más irregular con `aleatorio2` mostrando crecimiento acelerado en tiempo y comparaciones.

Asimismo, `ordenacion3` tiene consistencia con un crecimiento aparentemente lineal en todas las métricas y rellenos.

Las diferencias de escala entre algoritmos son visualmente impactantes, especialmente en `ordenacion1` donde `aleatorio2` reduce drásticamente las curvas de crecimiento.

3. Análisis de resultados

3.1. Métodos de análisis

Para determinar las fórmulas que mejor se ajustan a los datos experimentales, seguimos el siguiente procedimiento: consideramos tres tipos de funciones candidatas: n (lineal), n^2 (cuadrática), y $n \cdot \ln(n)$ (logarítmica lineal). Método de aproximación manual: Al estar utilizando excel, tuvimos que:

1. Crear columnas con los valores de n, n^2, y $n \cdot \ln(n)$ para cada tamaño.
2. Dividir los datos obtenidos (tiempo, asignaciones, comparaciones) entre cada una de estas funciones.
3. Si al dividir los datos entre una funcion el resultado se mantiene a simple vista constante, probablemente esa función sea la que describe el comportamiento del algoritmo.
4. Para verificar que esta sea la función que buscamos, calculamos el coeficiente de variación para cada una, y nos quedamos con la que su variación sea menor. Una variación pequeña indica que la función describe correctamente el crecimiento de los datos.
5. Para calcular la constante que acompaña a la función, calculamos el promedio de los cocientes.

3.2. Fórmulas obtenidas

3.2.1. Para `ordenacion1`

Con `aleatorio1`:
- Tiempo: $0,002 \cdot n^2$ (coef. variación: $0,75\,\%$)
- Asignaciones: $0,25 \cdot n^2$ (coef. variación: $0,10\,\%$)
- Comparaciones: $0,25 \cdot n^2$ (coef. variación: $0,08\,\%$)
- Orden asintótico: $\Theta(n^2)$

Con `aleatorio2`:
- Tiempo: $0,12 \cdot n$ (coef. variación: $2,82\,\%$)
- Asignaciones: $10,41 \cdot n$ (coef. variación: $0,11\,\%$)
- Comparaciones: $9,42 \cdot n$ (coef. variación: $0,05\,\%$)
- Orden asintótico: $\Theta(n)$

Este algoritmo es el de ordenación por inserción. Con `aleatorio1` genera un vector completamente desordenado, provocando el peor caso del algoritmo (n^2). Con `aleatorio2`, el vector queda casi ordenado ($k = 20$ limita el desplazamiento), por lo que el algoritmo se comporta casi linealmente, que es su mejor caso.

3.2.2. Para `ordenacion2`

Con `aleatorio1`:
- Tiempo: $0,032 \cdot n \cdot \ln(n)$ (coef. variación: $2,91\,\%$)
- Asignaciones: $1,09 \cdot n \cdot \ln(n)$ (coef. variación: $0,73\,\%$)
- Comparaciones: $2,22 \cdot n \cdot \ln(n)$ (coef. variación: $0,64\,\%$)
- Orden asintótico: $\Theta(n \cdot \ln(n))$

Con `aleatorio2`:
- Tiempo: $0,00022 \cdot n^2$ (coef. variación: $2,79\,\%$)
- Asignaciones: $4,97 \cdot n$ (coef. variación: $0,03\,\%$)
- Comparaciones: $0,056 \cdot n^2$ (coef. variación: $0,42\,\%$)
- Orden asintótico: $\Theta(n^2)$

Este algoritmo es el de ordenación rápida (quicksort) (siempre toma el primer elemento). Con `aleatorio1` funciona eficientemente porque el pivote tiene probabilidad de generar particiones balanceadas. Sin embargo, con `aleatorio2` el vector está casi ordenado, lo que hace que el pivote fijo siempre genere particiones muy desbalanceadas, llevando al peor caso (n^2) del quicksort.

3.2.3. Para `ordenacion3`

Con `aleatorio1`:
- Tiempo: $0,16 \cdot n \cdot \ln(n)$ (coef. variación: $5,96\,\%$)
- Asignaciones: $1,30 \cdot n \cdot \ln(n)$ (coef. variación: $1,67\,\%$)
- Comparaciones: $2,11 \cdot n \cdot \ln(n)$ (coef. variación: $0,83\,\%$)
- Orden asintótico: $\Theta(n \cdot \ln(n))$

Con `aleatorio2`:
- Tiempo: $0,15 \cdot n \cdot \ln(n)$ (coef. variación: $6,20\,\%$)
- Asignaciones: $7,17 \cdot n$ (coef. variación: $0,05\,\%$)
- Comparaciones: $2,10 \cdot n \cdot \ln(n)$ (coef. variación: $0,63\,\%$)
- Orden asintótico: $\Theta(n \cdot \ln(n))$

Este algoritmo es ordenación rápida con pivote aleatorio. Al elegir el pivote al azar, evita el peor caso incluso cuando el vector está casi ordenado. Mantiene el comportamiento $n \cdot \ln(n)$ en ambos tipos de relleno, demostrando ser el más robusto de los tres.

3.3. Comparación

3.3.1. Eficiencia según el tipo de relleno

Con `aleatorio1` (vector totalmente aleatorio):
- Más eficientes: `ordenacion2` y `ordenacion3` (ambas $\Theta(n \cdot \ln(n))$)
- Menos eficiente: `ordenacion1` ($\Theta(n^2)$)

Diferencia práctica: Para $n = 100{,}000$, `ordenacion2` tarda 36,2 ms mientras que ordenación1 tarda 19.418 ms (más de 500 veces más lento)

Con `aleatorio2` (vector casi ordenado):

- Más eficiente: `ordenacion1` ($\Theta(n)$). Aprovecha que el vector está casi ordenado.
- Intermedia: `ordenacion3` ($\Theta(n \cdot \ln(n))$). Mantiene buen rendimiento.
- Menos eficiente: `ordenacion2` ($\Theta(n^2)$). Se deteriora completamente.

Diferencia práctica: Para $n = 100{,}000$, `ordenacion1` tarda 11,7 ms, `ordenacion3` tarda 165,9 ms, y `ordenacion2` tarda 2.180 ms.

3.3.2. Análisis de robutez

Es `ordenacion3` quien demuestra ser la más consistente, con variaciones de solo un 5 % entre tipos de relleno ($n = 100{,}000$: 174,8 ms vs 165,9 ms). En contraste, `ordenacion1` varía 1.650 veces (11,7 ms vs 19.418 ms) y `ordenacion2` es impredecible, siendo el único algoritmo que empeora con datos casi ordenados (36,2 ms vs 2.180 ms).

4. Conclusiones

¿Cuál es el mejor algoritmo? Depende del contexto en el que nos encontremos, existe un algoritmo mejor según el caso:

- **Si sabemos que los datos están casi ordenados (es decir, usamos `aleatorio2`)**: usar `ordenacion1` (ordenación por inserción). Es el más rápido con diferencia en este escenario.
- **Si no sabemos cómo vienen los datos**: usar **ordenacion3** (ordenación rápida con pivote aleatorio). Es el más seguro y consistente, garantizando buen rendimiento en cualquier situación.
- **Si estamos seguros de que los datos están desordenados (es decir, usamos `aleatorio1`)**: `ordenacion2` es ligeramente más rápida que `ordenacion3`, pero el riesgo no merece la pena, es más seguro quedarse con `ordenacion3`.

Para un caso cualquiera, `ordenacion3` es la mejor opción porque garantiza buen rendimiento en cualquier situación. Por su parte, `ordenacion2` debe evitarse porque puede fallar catastróficamente con datos casi ordenados y puede generar desbordamiento de pila. A pesar de que `ordenacion1` muestra el orden más bajo, debe evitarse por el tiempo de ejecución que requiere en caso de estar los datos desordenados.

5. Porcentaje de trabajo individual

Consideramos que hemos distribuido el trabajo equitativamente entre ambas, con una participación del 50 % cada una.

Emily Fabiana Rodrigues Álvarez 50 %

Marta Pérez Alonso 50 %

6. Uso de inteligencia artificial generativa (IA-Gen)

Si bien se ha hecho uso de la inteligencia artificial generativa (IA-Gen), cada consulta y respuesta generadas fueron evaluadas antes de incluirlas en este análisis.

6.1. En la estructura del documento

En el desarrollo de este documento, se ha utilizado inteligencia artificial generativa (IA-Gen) como herramienta de apoyo para:

- Estructurar el documento mediante consultas para organizar las secciones de manera académica y coherente.
- Ayudarnos con el formato de LaTeX para la construcción de las tablas, así como para la adición de las gráficas.

En concreto, se consultó a `DeepSeek`, `ChatGPT` y `Gemini`.

6.2. En el código

En el código utilizado para generar los datos, se utilizó inteligencia artificial para la refactorización del mismo, específicamente:

- Para crear elementos de tipo `Enum` (enumerados de `Java`) para mejorar la legibilidad, que afectaba en la diferenciación de los tipos de algoritmos y métricas con los que trabaja el programa.
- Para crear la clase `Metricas`, que engloba la lógica de acumulador de datos de las métricas que luego son promediados por cada tamaño del vector.

También se hizo uso de la IA-Gen para la creación de los archivos `CSV` y el formateo de almacenamiento de datos en el mismo. Así como en la creación del `Makefile`.

En concreto, se consultó a `DeepSeek` y a `ChatGPT`.

6.3. En el cálculo de la complejidad

Se utilizó inteligencia artificial como apoyo durante el análisis en Excel, principalmente para obtener la fórmula del coeficiente de variación y aplicarla correctamente a los datos.

En concreto, se consultó a `Claude` y `ChatGPT`.

7. Bibliografía y Fuentes

Material de la asignatura

- Tema 1 - Análisis de la eficiencia de los Algoritmos. Departamento de Informática, Universidad de Valladolid. González Ferreras, César.

  Utilizado para: Fundamentos teóricos sobre análisis de algoritmos, notación asintótica, análisis del caso promedio, y reglas para medir algoritmos.

- Tema 2 - Diseño de Algoritmos. Departamento de Informática, Universidad de Valladolid. González Ferreras, César.

  Utilizado para: Estudio teórico de algunos algoritmos de ordenación implementados: ordenación rápida con pivote fijo y ordenación rápida con pivote aleatorio.